FREEBUFF SUPERPOWER ULTRA • TITANIUM SWARM V4.5

BUKU PANDUAN LENGKAP & DEEP DIVE ARSITEKTUR SUB-AGENTS & 1055 MODULAR SKILLS

Panduan Rinci dan Tajam Mengenai Mekanisme Paralelisasi, Orkestrasi 46 Sub-Agents Spesialis, Dual-Gate Quality Invariants, dan Taksonomi 1055 Modular Engineering Skills Tanpa AI-Slop.

46	1,055	9	Dual-Gate
SUB-AGENTS SPESIALIS	MODULAR SKILLS	KLASTER REKAYASA	QUALITY INVARIANTS

RINGKASAN KEMAMPUAN SISTEM:

- **Parallel Swarm Execution:** Meluncurkan N sub-agen simultan tanpa blocking.
- **Zero AI-Slop Manifesto:** Larangan keras kode placeholder `// TODO`, modul khayalan, dan filler korporat.
- **Just-In-Time (JIT) Skill Injection:** Memuat kapabilitas spesifik on-demand tanpa context bloat.
- **Standardized Inter-Agent Handoff:** Alur teruji `@architect` → `@programmer` → `@tddmaster` → `@security` → `memory-by-ferz`.

Versi Dokumen: **4.5 (Production Release)**
Lingkungan Proyek: **/home/firman/weblomba**

Standar Mutu: **Strict Anti-AI-Slop & Dual-Gate QA**
Orkestrator: **Titanium Swarm Autonomous Engine**

BAB I: RINGKASAN EKSEKUTIF & ARSITEKTUR TITANIUM SWARM

Inti Filosofi Sistem: Titanium Swarm v4.5 dirancang untuk mengatasi kelemahan mendasar model AI konvensional: *context window bloating*, degradasi kognitif saat tugas menumpuk, dan kemunculan *AI-slop* (kode setengah jadi yang tidak dapat dieksekusi). Dengan memecah beban ke dalam **46 Sub-Agents Spesialis** dan **1,055 Modular Skills**, setiap agen bekerja dengan fokus sempit berpresisi tinggi.

1.1 Mengapa Multi-Agent Swarm Paralel Diperlukan?

Dalam proyek perangkat lunak nyata, pengerjaan sekuensial (serial) oleh satu agen membuat waktu eksekusi membengkak dan menghasilkan konteks yang tercampur aduk (misal: kode frontend terdistorsi oleh aturan query database yang panjang). Titanium Swarm menerapkan pendekatan modular:

- **Isolasi Ruang Kerja (Workspace Isolation):** Setiap sub-agen memiliki context window, transcript, dan tools terpisah yang tidak saling mengotori.
- **Eksekusi Paralel Simultan:** Sub-agen audit keamanan, pengujian beban, dan penulisan dokumentasi berjalan pada detik yang sama di latar belakang.
- **Konsumsi Token Efisien:** Hanya sub-agen yang dibutuhkan yang dipanggil. Agen lain tetap berada dalam status *idle* tanpa memakan kuota token.

1.2 3-Tier Cognitive Depth Execution Matrix

Setiap instruksi secara otomatis diklasifikasikan ke dalam matriks kognitif untuk menentukan kedalaman perencanaan:

Tingkat Kerumitan	Kondisi Pemicu	Protokol Eksekusi	Jaminan Kualitas
Tier 1: Surgical	Typo fixes, styling CSS tweak, rename 1 variabel, patch cepat.	Eksekusi langsung secara bedah tanpa overhead dokumen RFC. Verifikasi linter/ build kilat.	Selesai dalam hitungan detik, zero planning bloat.
Tier 2: Feature Flow	Endpoint baru, komponen UI baru, migrasi skema tabel, utility helper.	Alur berantai: `@planner` (dekomposisi) → `@programmer` (implementasi) → `@tddmaster` (unit test).	Minimal 1 happy path test dan 2 negative edge case tests.
Tier 3: Deep Swarm	Refaktor arsitektur, auth system, ledger keuangan, sistem multi-service.	**War-Room Lengkap:** Wajib terbitkan `RFC.md` sebelum modifikasi kode. Verifikasi Dual-Gate QA.	100% Zero AI-Slop, lolos audit keamanan OWASP, zero regresi.

1.3 Dual-Gate Quality Invariants (Standar Mutu Mutlak)

Kode dilarang keras dianggap tuntas sebelum lulus kedua gerbang verifikasi berikut:

Gate 1: Static Quality
- Strict TypeScript (zero implicit any).
- Zero AI-Slop (tanpa `// TODO`, tanpa elipsis).
- Hallucination Circuit-Breaker (semua package diverifikasi ke registry).

Gate 2: Behavioral
- Verifikasi Happy-Path fungsional.
- Minimal 2 uji Negative Edge-Case (input invalid, expired token).
- Self-Healing Loop otomatis jika test gagal.

Gate 3: Memory Sync
- Penyimpanan state ke `memory/MEMORY.md` via `memory-by-ferz`.
- Preservasi keputusan arsitektur lintas sesi kerja.

BAB II: MEKANISME PARALELISASI & CARA KERJA SUB-AGENTS

Titanium Swarm memanfaatkan fitur **Multi-Agent Background Execution**. Bagian ini menguraikan bagaimana paralelisasi dihidupkan, dikontrol, dan disinkronisasikan.

2.1 Cara Kerja Teknis Paralelisasi

Paralelisasi sub-agen terjadi ketika orkestrator memanggil tool `invoke_subagent` dengan menyertakan beberapa agen sekaligus dalam satu array `Subagents`:

```
// Tool Call Pemanggilan 3 Sub-Agen Secara Simultan (Paralel)
invoke_subagent({
  Subagents: [
    {
      TypeName: "self",
      Role: "Security Auditor",
      Prompt: "Audit seluruh endpoint /api/v1 terhadap OWASP Top 10 dan laporkan celah otentikasi."
    },
    {
      TypeName: "self",
      Role: "QA Master",
      Prompt: "Tulis test suite Vitest untuk skenario happy path dan 3 kasus kegagalan ekstrim."
    },
    {
      TypeName: "self",
      Role: "Tech Writer",
      Prompt: "Susun panduan OpenAPI 3.1 dan sequence diagram untuk sistem pembayaran."
    }
  ]
});
```

Siklus Hidup Eksekusi Paralel (Parallel Lifecycle):

1. **Forking & Context Isolation:** Sistem membuat instance terpisah untuk masing-masing agen dengan ID percakapan independen.
2. **Simultaneous Background Running:** Agen-agen tersebut menjalankan shell commands, pembacaan file, atau analisis secara independen tanpa memblokir thread induk.
3. **Reactive Wakeup:** Parent agent tidak perlu melakukan polling (mengecek berulang). Sistem otomatis membangunkan parent agent melalui event callback saat pesan atau hasil kerja selesai dikirim.
4. **Synthesis & Arbitration:** Parent agent mengumpulkan hasil dari seluruh sub-agen, memecahkan potensi perbedaan analisis, dan memfinalisasi solusi.

2.2 Standar Protokol Handoff Antar Sub-Agen (Swarm Pipeline)

Untuk menghindari konflik modifikasi file pada eksekusi paralel, sistem menetapkan protokol alur kerja:

Tahap	Pelaksana	Aktivitas Teknis	Artefak Dokumen
Fase 1	`@architect` + `@planner`	Analisis kebutuhan, pembuatan interface types, DDL tabel.	`RFC.md` & `ADR.md`
Fase 2	`@programmer` + `@implementer`	Implementasi kode secara bedah, penulisan file, strictly-typed.	Source Code + `DIFF_LOG.md`
Fase 3	`@tddmaster` + `@qa`	Eksekusi unit test, integration test, fuzzing, dan regression suite.	`TEST_REPORT.md`
Fase 4	`@security` + `@reviewer`	Pemindaian kerentanan OWASP, kebocoran token/secret, eliminasi slop.	Security Sign-Off Certificate
Fase 5	`[Skill: memory-by-ferz]`	Sinkronisasi status arsitektur aktif ke memori persisten.	`memory/MEMORY.md`

2.3 Strategi Pencegahan Merge Conflict & Loop

- **Aturan Single-Writer:** Hanya satu agen yang memegang hak modifikasi (write) pada file tertentu dalam satu waktu. Sub-agen lain yang bekerja secara paralel hanya diberikan peran read-only (auditing, planning, drafting).
- **Autonomous Healer Circuit-Breaker:** Jika suatu unit test gagal setelah perubahan kode, sistem memicu `[Skill: autonomous-swe-loop-healer]` secara otomatis hingga maksimal 3 iterasi mandiri sebelum eskalasi.

BAB III: KATALOG LENGKAP 46 SUB-AGENTS SPESIALIS (1/2)

Berikut rincian 46 Sub-Agents resmi dalam `.freebuff/agents/` yang dikelompokkan ke dalam 9 klaster rekayasa sistem:

1. Architecture & System Design (7 Agen)

Nama Agen	Peran & Spesialisasi	Kasus Penggunaan Optimal
`@programmer-omni` **SUPREME COMMANDER**	Panglima tertinggi swarm dengan akses 66 skills, otoritas dispatching penuh, dan penegakan strict anti-AI-slop.	Proyek baru berskala besar, koordinasi multi-service, refaktor kritis.
`@architect` **SYSTEM DESIGN**	Dekomposisi First-Principles, merancang topology sistem, DDL database terstruktur, dan interface boundaries.	Penyusunan formal `RFC.md` sebelum coding dimulai.
`@planner` **TASK DECOMPOSITION**	Memecah requirement besar menjadi sub-tugas atomik berbasis DAG (Directed Acyclic Graph) tanpa mengubah kode.	Manajemen dependensi tugas teknis dan roadmap implementasi.
`@monorepo-architect` **MONOREPO**	Arsitek monorepo enterprise: Turborepo, Nx, PNPM workspaces, package boundary isolation, dan shared configs.	Pemisahan paket multi-library dan manajemen monorepo.
`@microservices-mesh` **MICROSERVICES**	Arsitektur terdistribusi: Istio Service Mesh, gRPC HTTP/2, Kafka event streams, dan fault tolerance patterns.	Komunikasi antar-service, traffic routing, microservices.
`@explore-plus` **CODEBASE EXPLORER**	Pemetaan cepat struktur kode asing: entry points, call graphs, dan dependensi tersembunyi.	Onboarding repositori warisan (legacy) atau codebase baru.
`@researcher` **DEEP RESEARCH**	Riset otonom: ekstraksi dokumentasi resmi, paper ilmiah, benchmark teknologi, dan perbandingan solusi.	Evaluasi teknologi baru, pemilihan framework, analisis CVE.

2. Frontend & UI/UX Experience (8 Agen)

Nama Agen	Peran & Spesialisasi	Kasus Penggunaan Optimal
`@frontend` **MODERN WEB**	Frontend modern: React 19, Next.js 15 App Router, Tailwind CSS, shadcn/ui dengan pendekatan anti-slop.	Pembangunan komponen web responsif dan state management.
`@design-engineer` **DESIGN SYSTEM**	Penerjemah token brand, visual micro-interactions, OKLCH color token, dan tipografi Swiss/Editorial.	Pembangunan design system eksklusif tanpa kesan template AI.
`@generative-ui-engineer` **GENUI & STREAMING**	Streaming React Server Components (RSC) via Vercel AI SDK, client WebGPU LLMs, dan generative tool-calling UI.	Aplikasi obrolan AI canggih dan widget visual streaming.
`@game-dev-3d` **3D & WEBGL**	Grafika 3D interaktif: Three.js, WebGL, custom GLSL shaders, React Three Fiber (R3F), engine fisika Rapier.	Pengalaman 3D interaktif 60 FPS langsung di browser web.
`@accessibility-champion` **WCAG 2.2 AAA**	Kepatuhan aksesibilitas tinggi: trap fokus keyboard, screen reader live regions (`aria-live`), rasio kontras.	Audit website inklusif dan sertifikasi aksesibilitas formal.
`@localization-i18n-pro` **I18N & RTL**	Internasionalisasi enterprise: routing dinamis multilingual, layout RTL (Arab/Ibrani), ICU message strings.	Aplikasi global multibahasa dengan dukungan teks dua arah.
`@seo-growth` **PSEO & VITALS**	Programmatic SEO, Schema.org JSON-LD, kartu OpenGraph dinamis, optimasi Core Web Vitals (LCP/INP).	Peningkatan ranking pencarian Google dan GEO AI citation.
`@pwa-offline-architect` **PWA OFFLINE**	Progressive Web App: service worker caching (Workbox), sync offline background, web push notifications.	Aplikasi web yang dapat berfungsi penuh tanpa jaringan internet.

3. Backend & Database Engineering (6 Agen)

Nama Agen	Peran & Spesialisasi	Kasus Penggunaan Optimal
`@backend` **APIS & AUTH**	REST/gRPC APIs aman, sesi HttpOnly, rate limiting distributed, validasi skema Zod/Pydantic.	Pembangunan fondasi server, middleware, dan lifecycle auth.

Nama Agen	Peran & Spesialisasi	Kasus Penggunaan Optimal
`@database` **DATABASE ARCHITECT**	PostgreSQL internals, indexing B-Tree/GIN, query plan tuning, zero-downtime expand-contract migration.	Pembersihan query lambat dan perancangan skema data besar.
`@data-engineer-olap` **OLAP & STREAMING**	Analitik real-time: ClickHouse, DuckDB SQL in-process, format data Parquet, ingestion jutaan data/detik.	Dashboard analitik data masif dengan respon sub-detik.
`@websocket-realtime` **REALTIME & SSE**	Koneksi dua arah: WebSockets, Server-Sent Events, Redis Pub/Sub cluster, presence tracking.	Chat multi-user, kolaborasi dokumen real-time, live feeds.
`@fintech-architect` **FINTECH PAYMENTS**	Siklus pembayaran Stripe/Xendit, verifikasi webhook signature, idempotency replay cache.	Sistem checkout pembayaran dan pencegahan double-charge.
`@tokenomics-fintech-ledger` **ACID LEDGERS**	Pembukuan Double-Entry berstandar akuntansi, transaksi ACID mutlak, audit trails immutable.	Dompet digital, sistem perbankan terdesentralisasi, saldo saldo.

BAB III: KATALOG LENGKAP 46 SUB-AGENTS SPESIALIS (2/2)

4. Core Implementation & Craftsmanship (3 Agen)

Nama Agen	Peran & Spesialisasi	Kasus Penggunaan Optimal
`@programmer` **SUPER PROGRAMMER**	Ultimate Super Programmer v3.5: rekayasa bedah satu arah, kode bersih standar manusia, dan sync memory-by-ferz.	Penulisan kode produksi inti yang ringkas, cepat, dan rapi.
`@implementer` **FULLSTACK EXEC**	Eksekusi fitur end-to-end berpegang pada RFC dengan git diff minimal dan verifikasi build instan.	Implementasi fitur komplit frontend + backend dalam 1 siklus.
`@refactor-expert` **AST & MODERNIZE**	Refactoring skala besar menggunakan Abstract Syntax Tree (AST), pembersihan code smell, dan Clean Architecture.	Migrasi modul usang tanpa mengubah fungsionalitas (zero regression).

5. Mobile & Cross-Platform (2 Agen)

Nama Agen	Peran & Spesialisasi	Kasus Penggunaan Optimal
`@mobile-engineer` **MOBILE CROSS-PLATFORM**	React Native (Expo) & Flutter: gestur sentuh, bottom-sheets bersuspensi fluida, background sync, notifikasi push.	Aplikasi iOS dan Android native berkinerja tinggi 60 FPS.
`@flutter-specialist` **FLUTTER DEEP**	Spesialis Flutter murni: state management BLoC/Riverpod, CustomPainter animasi kompleks, native platform channels.	Aplikasi Flutter dengan UI kustom dan performa rendering native.

6. AI, NLP & Cognitive Systems (5 Agen)

Nama Agen	Peran & Spesialisasi	Kasus Penggunaan Optimal
`@ai-engineer` **AI SYSTEMS**	Infrastruktur RAG: pgvector/Qdrant, evaluasi prompt LLM, routing token cerdas, dan local LLM deployment (Ollama).	Pembuatan pipeline pencarian semantik dan integrasi LLM.
`@nlp-rag-specialist` **ADVANCED RAG**	Sistem retrieval presisi: hybrid search (BM25 + vector), semantic chunking, Cohere Rerank v3, dan HyDE.	Knowledge base korporat dengan akurasi pencarian tinggi.
`@deepseek-reasoner-pro` **DEEP REASONING**	Penalaran matematika murni, optimasi algoritma komputasi rumit, dan pembuktian invarian sistem.	Pemecahan algoritma graf, optimasi komputasional, verifikasi formal.
`@glm-chat` **IDEA BRAINSTORM**	Konsultasi ide kilat (144 tok/s), eksplorasi alternatif solusi, dan brainstorming arsitektur.	Fase perumusan ide dan diskusi opsi teknis di awal proyek.
`@prompt-red-teamer` **AI RED TEAMING**	Uji penetrasi AI: penangkal prompt injection adversarial, jailbreak defense, indirect data poisoning, guardrails.	Pengerasan keamanan prompt dan tool calling sebelum produksi.

7. DevOps, Cloud Native & SRE (4 Agen)

Nama Agen	Peran & Spesialisasi	Kasus Penggunaan Optimal
`@devops` **CI/CD & DOCKER**	Docker multi-stage aman (<50MB), pipeline GitHub Actions, Terraform IaC, dan otomasi bash POSIX.	Penyusunan alur build otomatis dan infrastruktur container.
`@cloud-native-devops-k8s` **KUBERNETES**	Kubernetes enterprise: Helm charts, ArgoCD GitOps, autoscaling HPA/KEDA, dan deployment zero-downtime.	Orkestrasi klaster K8s dan manajemen rilis terdistribusi.
`@sre` **SRE & METRICS**	Site Reliability Engineering: SLO/SLI budgets, Prometheus/ Grafana alerts, dan mitigasi downtime sistem.	Pemeliharaan keandalan sistem produksi 99.99% dan triage insiden.
`@release` **RELEASE MANAGEMENT**	Manajemen rilis: version bump otomatis SemVer, pembuatan changelog, dan publishing GitHub Release tags.	Finalisasi pengiriman perangkat lunak ke publik.

8. Security, Governance & Auditing (5 Agen)

Nama Agen	Peran & Spesialisasi	Kasus Penggunaan Optimal
`@security` **OWASP SECURITY**	Audit OWASP Top 10, scanning kebocoran kredensial (.env), hardening CSP, dan kepatuhan IAM.	Pemeriksaan celah kerentanan aplikasi sebelum deployment.
`@cloud-security-architect` **ZERO TRUST CLOUD**	Arsitektur Zero-Trust: AWS/GCP IAM least-privilege, segmentasi VPC, mutual TLS, dan manajemen Vault.	Hardening keamanan cloud dan audit akses infrastruktur.
`@smart-contract-auditor` **WEB3 & SOLIDITY**	Audit smart contract EVM/Solidity: pencegahan reentrancy, optimasi gas Yul, dan fuzzing Foundry.	Audit keamanan protokol blockchain dan tokenomics.
`@reviewer` **CODE REVIEWER**	Tinjauan kualitas independen: mendeteksi bug tersembunyi, anti-pattern, dan memverifikasi kesiapan PR.	Code review objektif sebelum merge kode ke branch utama.
`@writer` **TECHNICAL WRITER**	Penulisan dokumentasi teknis: README komprehensif, panduan API OpenAPI, dan format dokumen resmi.	Penyusunan dokumentasi yang sinkron dengan kode sumber nyata.

9. Performance, QA & Forensics Testing (6 Agen)

Nama Agen	Peran & Spesialisasi	Kasus Penggunaan Optimal
`@tddmaster` **TDD MASTER**	Metodologi Red-Green-Refactor murni: test suite deterministik, edge case coverage, dan mock isolation.	Menulis test suite komprehensif sebelum modul diimplementasi.
`@qa` **E2E AUTOMATION**	Otomasi Playwright end-to-end, simulasi alur multi-role pengguna, dan pelaporan cacat perangkat lunak.	Pengujian alur pengguna dari browser secara otomatis.
`@chaos-tester` **CHAOS & LOAD**	Chaos engineering: simulasi kegagalan jaringan, pengujian beban ekstrem Grafana k6, dan circuit breakers.	Pengujian ketahanan sistem saat terjadi bencana trafik atau server mati.
`@perf` **PERFORMANCE**	Optimasi latensi: profiling flamegraph CPU, memory allocation, dan perbaikan metrik Core Web Vitals.	Memangkas response time lambat dan lag animasi.
`@debugger` **BUG TRIAGE**	Isolasi dan reproduksi bug secara deterministik, penelusuran akar masalah, dan perbaikan minimal presisi.	Memperbaiki bug rumit tanpa mengganggu bagian kode lain.
`@system-debugger-forensics` **FORENSICS EBPF**	Forensik level rendah: tracing Linux kernel via eBPF, analisis thread lock, dan dump heap memory.	Investigasi crash misterius di level OS atau runtime native.

BAB IV: SISTEM 1,055 SKILLS & SPOTLIGHT 28 FLAGSHIP SKILLS

Titanium Swarm memiliki perbendaharaan **1,055 Keterampilan Rekayasa Modular (Skills)** yang disimpan di `.freebuff/skills/`. Setiap modul berisi aturan deterministik, SOP rekayasa teruji, dan skrip otomasi.

4.1 Peta Distribusi 9 Pilar Rekayasa

Berdasarkan analisis kurasi mendalam, seluruh 1,055 skills terdistribusi ke dalam 9 domain spesifik:

- **Frontend & UI Design (315+ skills):** Menguasai React, Tailwind v4, Radix Primitives, Canvas 60 FPS, micro-interactions, dan WCAG AAA.
- **Backend & APIs (102+ skills):** Runtime Bun, Go concurrency, microservices, gRPC HTTP/2, GraphQL Federation, dan transaksi idempoten.
- **Automation & Tooling (87+ skills):** Skrip POSIX bash, AST codemods Tree-sitter, sanitasi kode otomatis, dan SDD templates.
- **Workflow, Memory & DX (78+ skills):** Persistent memory-by-ferz, automated SWE loop healing, konvensi commit, dan audit DX.
- **Architecture & Clean Code (43+ skills):** First-principles reasoning, state machines, Chesterton's fence, dan anti-overengineering.
- **Security & Invariants (38+ skills):** OWASP Top 10, Passkeys WebAuthn, pemindaian SAST/DAST, dan audit smart contract EVM.
- **Data & Storage (37+ skills):** PostgreSQL internals, ClickHouse OLAP, DuckDB, Redis streams, dan migrasi expand/contract.
- **Testing & Quality (33+ skills):** TDD red-green, k6 load testing, Playwright visual diffs, dan mutation testing Stryker.
- **DevOps & Cloud (18+ skills) & Mobile (13+ skills):** K8s Helm, Cloudflare KV edge, Flutter BLoC, dan PWA offline caching.

4.2 Kurasi 28 Flagship Skills Paling Transformatif

Tabel berikut menyajikan 28 flagship skills revolusioner yang dapat diaktifkan menggunakan tag `[Skill: <nama>]` atau shortcut terkait:

Nama Skill & Jalur	Fungsi Teknis & Mekanisme	Dampak Rekayasa Sistem
`autonomous-swe-loop-healer` **SHORTCUT: /HEAL**	Otomatis mendiagnosis stack trace, merumuskan hipotesis, menerapkan patch, dan menjalankan ulang test hingga lulus.	**Self-healing bugfix mandiri** tanpa intervensi manual manusia.
`memory-by-ferz` **SHORTCUT: /MEMORY**	Mesin memori persisten poliglot di `memory/MEMORY.md` yang merekam arsitektur aktif dan skema DB.	**Zero context amnesia**; progres tetap terjaga sempurna lintas sesi.
`hallmark` **TAG PROMPT**	Standar desain UI anti-slop: evaluasi P5 H4 E5 S4 R5 V5, warna OKLCH, larangan kartu ungu-neon klise buatan AI.	Antarmuka berkelas dunia setara karya desainer produk papan atas.
`no-hallucinated-dependencies` **TAG PROMPT**	Circuit-breaker yang memverifikasi setiap package impor ke registri publik resmi sebelum dieksekusi.	**Nol impor fiktif**; mencegah build error fatal di CI/CD.
`kill-ai-slop` **SHORTCUT: /REVIEW**	Menghapus basa-basi percakapan, placeholder `// TODO`, komentar restatement, dan elipsis kode.	Menghasilkan kode produksi yang 100% tuntas dan bermutu tinggi.
`agentic-rag-hybrid-reranking-cohere` **TAG PROMPT**	Multi-stage retrieval: Reciprocal Rank Fusion (vector + BM25) → HyDE → Cohere Rerank v3 filtering.	Presisi pencarian semantik dokumen mencapai akurasi > 95%.
`spec-driven-development-sdlc` **SHORTCUT: /PLAN**	Mewajibkan pembuatan formal `RFC.md` dan DDL skema sebelum menulis kode implementasi.	Mengeliminasi salah arah arsitektur dan menghemat waktu rework.
`state-machine-invariant-reasoning` **TAG PROMPT**	Memodelkan alur kerja menjadi Finite State Machine (FSM); membuat state ilegal mustahil terjadi pada level tipe data.	Mencegah race condition dan transisi data tidak sah secara matematis.
`deterministic-code-sanitizer` **TAG PROMPT**	Memindai dan memusnahkan stub palsu, dummy mock, dan empty error handlers secara otomatis dari codebase.	Kode bersih dari artefak prototipe yang rentan jebol di produksi.
`binary-search-debugging-isolate` **TAG PROMPT**	Menemukan commit dan baris penyebab regresi dalam $O(\log n)$ langkah menggunakan git bisect otomatis.	Isolasi akar penyebab bug misterius dalam hitungan menit.

Nama Skill & Jalur	Fungsi Teknis & Mekanisme	Dampak Rekayasa Sistem
`bun-runtime-hyper-fast-apis` TAG PROMPT	Membangun backend microservices dan WebSockets menggunakan native `Bun.serve()` dan embedded SQLite.	Throughput API hingga 30x lebih kencang dengan cold-start minimal.
`idempotency-distributed-transactions` TAG PROMPT	Pola transaksi idempoten via header `Idempotency-Key`, Redis distributed locks, dan replay cache.	Keamanan mutlak transaksi keuangan dan antrean distributed messaging.
`postgresql-internals-indexing-master` TAG PROMPT	Optimasi engine internal Postgres: indeks B-Tree, GIN, BRIN, analisis EXPLAIN BUFFERS, dan autovacuum tuning.	Query berkecepatan sub-milidetik pada database volume terabyte.
`prompt-injection-defense-guardrails` TAG PROMPT	Membentengi LLM dari adversarial prompt injection, jailbreak, dan indirect poisoning via NeMo Guardrails.	Perlindungan maksimal bagi agen AI yang menerima input publik bebas.
`k6-spike-soak-breakpoint-testing` TAG PROMPT	Pengujian beban Grafana k6: lonjakan trafik 10x, soak test memori 24 jam, dan penemuan limit breakpoint kapasitas.	Mengetahui daya tampung sistem secara eksak sebelum launching.
`playwright-visual-regression-testing` TAG PROMPT	Uji regresi visual pixel-perfect: membandingkan diff screenshot multi-viewport dan mode Light/Dark.	Nol pergeseran layout (CLS) atau cacat visual yang lolos ke staging.
`mutation-testing-stryker-mutmut` TAG PROMPT	Menyuntikkan mutasi bug sengaja ke kode untuk menguji apakah unit test benar-benar efektif mendeteksi eror.	Memvalidasi kualitas hakiki dari test suite tanpa false confidence.
`zero-downtime-migration-expand-contract` TAG PROMPT	Evolusi skema database: Expand (tambah kolom baru) → Dual-write → Backfill → Contract (hapus kolom lama).	Migrasi skema database aman dengan ketersediaan sistem 99.99%.
`crdt-realtime-collaboration-yjs` TAG PROMPT	Sinkronisasi data multi-user kolaboratif (Figma/Notion-like) dengan Yjs CRDTs dan awareness cursor sync.	Kolaborasi real-time tanpa resiko penimpaan data (merge conflicts).
`distributed-tracing-opentelemetry` TAG PROMPT	Pelacakan request end-to-end terdistribusi dengan OpenTelemetry (OTel), context propagation W3C, dan Jaeger.	Visibilitas penuh latensi mikroservice dan deteksi bottleneck instan.
`bento-grid-dashboard-ui` TAG PROMPT	Desain dashboard modern bento-grid asimetris dengan dynamic masonry spans dan kartu glassmorphic.	Penyajian metrik analitik yang padat informasi, intuitif, dan mewah.
`accessible-wcag-aaa-keyboard-aria` TAG PROMPT	Penerapan kepatuhan WCAG 2.2 AAA: keyboard focus trap, aria-live announce regions, dan validasi rasio kontras.	Antarmuka yang dapat dioperasikan penuh oleh pengguna disabilitas.

BAB V: PANDUAN PRAKTIS & CHEAT SHEET OPERASIONAL

Bab ini memberikan instruksi langsung dan contekan sintaks praktis untuk mengoperasikan Sub-Agents dan Skills secara maksimal di terminal maupun antarmuka chat harian.

5.1 Panduan Cepat Perintah Slash (Command Palette)

Perintah Shortcut	Sub-Agent / Skill yang Aktif	Tindakan Teknis Otomatis
`/plan <task>`	`@architect` + `@planner`	Menganalisis kebutuhan, mendesain interface & DDL tabel, menyusun `RFC.md` sebelum koding.
`/build <feature>`	`@programmer` + `@implementer`	Mengeksekusi pembuatan fitur end-to-end secara strictly typed dengan standar anti-AI-slop.
`/test <target>`	`@tddmaster` + `@qa`	Menulis dan menjalankan test suite komprehensif (minimal 1 happy-path dan 2 edge cases).
`/heal`	`[Skill: autonomous-swe-loop-healer]`	Mendeteksi error build/runtime, mengisolasi akar masalah, menerapkan patch, dan menguji ulang.
`/review`	`@reviewer` + `@security`	Mengaudit kebenaran logika, scanning celah OWASP, memeriksa integritas tipe data, dan membersihkan slop.
`/memory`	`[Skill: memory-by-ferz]`	Menyimpan ringkasan arsitektur aktif dan kemajuan proyek ke `memory/MEMORY.md`.

5.2 Tiga Metode Mengaktifkan Sub-Agents & Skills

Metode 1: Tag Mention (@) & Skill Tag dalam Prompt Chat

Cukup sebutkan peran agen dan skill yang diinginkan dalam instruksi natural Anda:

```
"Tolong @architect rancang sistem antrian pemesanan tiket dengan [Skill: spec-driven-development-sdlc].
Lalu @programmer implementasikan menggunakan [Skill: bun-runtime-hyper-fast-apis] dan pastikan
transaksinya aman dengan [Skill: idempotency-distributed-transactions]."
```

Metode 2: Aktivasi Paralel Multi-Agent (Prompt Simultan)

Perintahkan beberapa agen bekerja bersamaan dalam satu prompt instruksi:

```
"Jalankan secara PARALEL sekarang:
1. @security: Audit file auth.ts terhadap celah session hijacking dan CSRF.
2. @qa: Buat test suite Vitest untuk mensimulasikan token kedaluwarsa.
3. @writer: Susun dokumentasi alur login OAuth2 untuk tim frontend."
```

Metode 3: Aktivasi Paralel via API Tool Call (Otomasi Terprogram)

```
{
  "name": "invoke_subagent",
  "args": {
    "Subagents": [
      {
        "TypeName": "self",
        "Role": "Database Optimizer",
        "Prompt": "Analisis query lambat pada tabel transaksi dan rekomendasikan indeks BRIN/GIN."
      },
      {
        "TypeName": "self",
        "Role": "Security Specialist",
        "Prompt": "Pindai seluruh dependency di package.json menggunakan [Skill: no-hallucinated-dependencies]."
      }
    ]
  }
}
```

5.3 Ringkasan Praktik Terbaik (Anti-Failure Rules)

5 ATURAN EMAS OPERASIONAL TITANIUM SWARM:

1. **Jangan Pernah Gunakan Dummy Code:** Larangan keras menulis `// TODO` atau mengabaikan blok catch. Semua kode harus fungsional nyata.
2. **Cegah Halusinasi Dependensi:** Selalu cek `package.json` sebelum mengimpor pustaka baru; jangan mengarang nama package.
3. **Satu Penulis Kode (Single Writer):** Dalam alur paralel, hanya satu agen yang boleh memodifikasi file target untuk mencegah konflik merge.
4. **Lakukan Sync Memori Rutin:** Ketik `/memory` sebelum mengakhiri sesi kerja agar konteks penting tidak lenyap di sesi berikutnya.
5. **Jalankan Uji Edge-Case:** Keberhasilan satu skenario happy path belum cukup; selalu sertakan 2 kasus pengujian ekstrem (input kosong, token expired).

Sistem: **Freebuff Superpower Ultra v4.5** Status: **Verified & Certified Production Architecture** Lokasi: **/home/firman/weblomba**